



Smart Contract Security Audit

Protocol Assessment: PasswordStore

Lead Security Auditor: G.N.V.Vinay Sankar
Date of Assessment: May 19, 2026
Report Version: 1.0.0
Target Blockchain: Ethereum

1 Executive Summary

1pt

This report details the security review of the `PasswordStore` smart contract. The contract is designed to let users securely save a password on the blockchain and retrieve it later. According to the project rules, only the contract owner should be allowed to view or change this password.

1.1 Audit Summary & Findings Breakdown

The review found critical issues that break the main goals of the contract. Specifically, the stored password is not actually a secret, and anyone can change it. The complete breakdown of the findings is listed below:

Vulnerability Risk Tier	Identified Instances
High Risk	2
Medium Risk	0
Low Risk	0
Informational	1

Table 1: Security Vulnerability Summary Table

1.2 Audit Scope Details

The security review evaluated the following codebase files:

Metric	Audit Context
Commit Hash	2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
Compiler Version	Solc 0.8.18
Reviewed File	./src/PasswordStore.sol

Table 2: Scope Specifications

Target: PasswordStore::s_password State Variable

Everything saved inside an Ethereum smart contract is completely public. Marking the `s_password` variable as `private` does not hide it from the public; it only stops other smart contracts from reading it during runtime. Anyone running a standard blockchain tool can read the raw data directly out of the contract's storage slots from outside the blockchain.

There is zero privacy. Anyone can look up the contract on the network and see the plain-text password, making the contract completely useless.

You can easily prove that the data is visible by running these commands on a local test network:

1. Start a local test network using Anvil:

```
make anvil
```

2. Deploy the contract to your local network:

```
make deploy
```

3. Read the exact storage slot (Slot 1) where the password variable is kept:

```
cast storage <DEPLOYED CONTRACT ADDRESS> 1 --rpc-url http
://127.0.0.1:8545
```

This returns the raw hex data:

[illegible]

4. Convert that hex data back into a readable text string:

[illegible]

The final output reveals the secret password in plain text:

```
myPassword
```

Change the design of the application. If you must store a password on the blockchain, encrypt it first off-chain (on the user's computer) before sending it to the network. This means the user will need to manage a separate key to decrypt it later. You should also completely remove the public view function so that users do not accidentally leak secrets into transaction histories.

[H-2] PasswordStore::setPassword has no access control, meaning a non-owner could change the password

Severity: High Risk

Target: PasswordStore::setPassword() Function

Description

The `setPassword` function is marked as `external`, which means anyone in the world can call it. However, the developer wrote a code comment (NatSpec) stating that *“This function allows only the owner to set the new password.”* Even though this was the goal, the actual code lacks any checks or restrictions to verify if the caller is the owner.

```
function setPassword(string memory newPassword) external {
    /// @custom:audit Missing restriction - there is no check for the
    owner
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact

Any random user or malicious actor can call this function and overwrite the stored password with their own, completely breaking the contract’s intended logic.

Proof of Concept (PoC)

We can write a simple test case in `PasswordStore.t.sol` to show that an unauthorized user can overwrite the password successfully:

```
function test_anyone_can_set_password(address randomAddr) public {
    vm.assume(randomAddr != owner);
    vm.prank(randomAddr); // Act as a random address

    string memory newPassword = "newPassword";
    passwordStore.setPassword(newPassword);

    vm.prank(owner);
    string memory actualCurrentPassword = passwordStore.getPassword();
    assertEquals(actualCurrentPassword, newPassword); // The password was
    changed successfully!
}
```

Running the test suite confirms that this unauthorized change passes without errors:

```
forge test --mt test_anyone_can_set_password
Compiler run successful!

Ran 1 test for test/PasswordStore.t.sol:PasswordStoreTest
[PASS] test_anyone_can_set_password(address) (runs: 256, u: 25712, ~:
25712)
```

Recommended Mitigation

Add a check at the very beginning of the function to make sure only the owner can proceed:

```
if (msg.sender != owner) revert PasswordStore__NotOwner();
```

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that does not exist, causing the natspec to be incorrect

Severity: Informational

Target: Documentation Comment Block

Description

The documentation comment written right above the `getPassword` function describes a variable parameter named `newPassword`. However, the function itself takes no inputs at all.

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory)
```

Impact

This is a minor documentation error. It doesn't break the code, but it can confuse developers who are reading the project files.

Recommended Mitigation

Simply delete the incorrect documentation line:

```
- * @param newPassword The new password to set.
```

3 Disclaimer

1pt

This smart contract security review is provided on an “as-is” basis. While this audit points out key issues discovered during the review, it cannot completely guarantee absolute safety against future bugs, platform vulnerabilities, or compiler updates.